

Fuerza bruta

Un algoritmo de fuerza bruta consiste en generar todas las soluciones factibles y quedarse la mejor.

En general, tiene complejidad exponencial.

Para problemas de búsqueda en un conjunto S .

Queremos hacer algo con los elementos que cumplen una cierta propiedad P . Podemos recorrer todo S evaluando P en cada elemento.

La complejidad en general será $\Omega(|S|)$

esquema:

for $x \in S$ do :

if $P(x)$:

procesar x

Backtracking

Es una técnica para generar espacios de búsqueda recursivos.

Lo hace mediante la extensión de soluciones parciales.

Ejemplo: Dado un conjunto $S = \{n_1, n_2, \dots, n_N\}$ y un valor T . Determinar si $\exists$ un subconjunto de S cuya suma sea $= T$.
 $x \in \{$. Si $T = 12$ y $S = \{6, 12, 6\} \Rightarrow$ una solución válida es el subconjunto $\{6, 6\}$.

espacio de búsqueda $\rightarrow$ todos los subconjuntos de S . Hay 2^N

Es una modificación del algoritmo de fuerza bruta, para evitar analizar todas las posibilidades.

Análisis de complejidad en árboles de decisión.

número total de nodos $\rightarrow O(2^N)$

operaciones por nodo $\rightarrow$ una cantidad cte para verificar la suma y decidir la poda.

complejidad temporal $\rightarrow O(2^N)$

complejidad espacial $\rightarrow O(N)$

Poda: criterio para cortar ramas y encontrar una solución (bajarle la complejidad al problema).

↳ Factibilidad: detener la búsqueda si la suma parcial supera T .

↳ Optimalidad: si alcanzamos T antes de considerar todos los elementos, detenernos esa rama.

Programación Dinámica

Se divide el problema en subproblemas de tamaños menores que se resuelven recursivamente.

Teorema: si $n \geq 0$ y $0 \leq k \leq n$, entonces:

$$\binom{n}{k} = \begin{cases} 1 & \text{si } k=0 \vee k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{si } 0 < k < n \end{cases}$$

Superposición de estados: el árbol de llamadas recursivas resuelve el mismo problema varias veces. Se realizan varias llamadas a la función recursiva con los mismos parámetros.

Para evitar estas repeticiones :

- ↳ **Top-down**: se implementa recursivamente, pero se guarda el resultado de cada llamada recursiva en una estructura de datos (memoización). Si una llamada recursiva se repite, se toma el resultado de esta estructura.
- ↳ **bottom-up**: resolvemos primero los subproblemas más pequeños y guardamos todos los resultados (en gral en una tabla).

Superposición de problemas: ocurre si la cantidad de llamadas recursivas de una función sin memorizar es mucho mayor que la cantidad de estados posibles.
subproblemas

Para calcular la complejidad, sumamos la complejidad de cada estado por el que pasamos.
 n estados posibles ($n+1$). En cada estado hacemos $O(1)$ operaciones.

costo de inicializar la memoria : $O(n)$.

Backtracking con Fibonacci

Función:

$$\text{fib}(n) = \begin{cases} 0 & \text{si } n=0 \\ 1 & \text{si } n=1 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{si no} \end{cases}$$

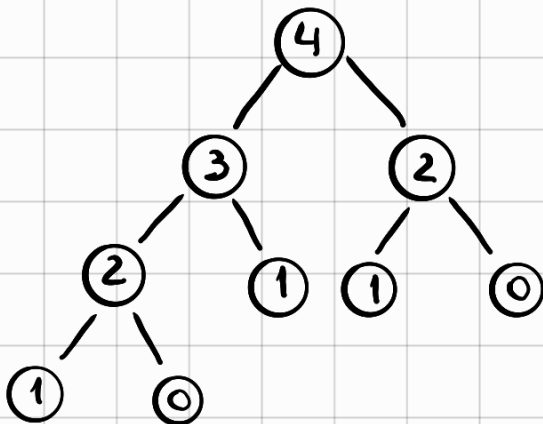
Código:

```
def fib(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

complejidad temporal: $O(2^n)$
 $\sqrt{2} (2^{n/2})$

complejidad espacial: $O(n)$

árbol:



Top-down:

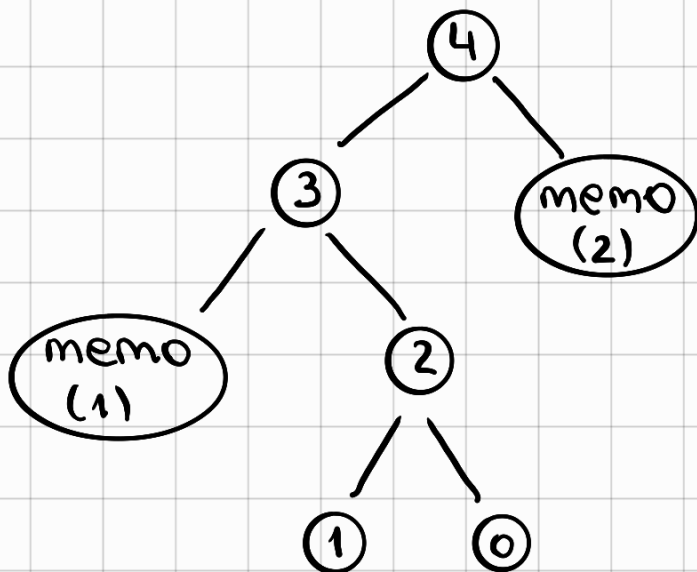
```
def fib(n: int, memo: list = None) → int:  
    if memo is None:  
        memo = [-1] * (n+1)  
    if memo[n] == -1:  
        if n ≤ 1:  
            memo[n] = n  
        else:  
            memo[n] = \  
                fib(n-1, memo) + fib(n-2, memo)  
    return memo[n]
```

Complejidad temporal:

$$\begin{array}{l} O(2n) = O(n) \\ \Omega(2n) = \Omega(n) \end{array} \quad \left. \vphantom{\begin{array}{l} O(2n) = O(n) \\ \Omega(2n) = \Omega(n) \end{array}} \right\} \theta(2n) = \theta(n)$$

Complejidad espacial: $O(n)$

Árbol:



Bottom-up: arranco guardando los primeros

```
def fib(n: int) → int:
    memo = [-1] * (n+1)
    memo[0] = 0
    memo[1] = 1
    for i in range(2, n+1):
        memo[i] = memo[i-1] + memo[i-2]
    return memo[n]
```

complejidad temporal: $O(n)$

complejidad espacial: $O(n)$

Diferencias:

- ↳ **Top-down:**
 - recursivo en Gral
 - más Fácil de programar (agarrás backtracking y le agregás memoización).
- ↳ **bottom-up:**
 - iterativo en Gral
 - usa menos memoria
 - es más rápido.

Principio de Optimalidad

Las partes de una solución óptima a un problema, deben ser soluciones óptimas de los correspondientes subproblemas.

correctitud de camino mínimo: $\text{minCamino}(\text{fila}, \text{col}, M)$ es el costo de un camino mínimo desde $(\text{fila}, \text{col})$ hasta (m, n) en la matriz M para $\text{fila} \in \{0, \dots, m\}$ y $\text{col} \in \{0, \dots, n\}$

Divide and conquer

ej: búsqueda binaria

Se basa en dividir un problema en subproblemas del mismo tipo que el original, resolver los problemas más pequeños (conquistar) y combinar las soluciones.

forma genl del D&C: dado $f(x)$. Divido a x en $x_1, x_2, \dots, x_k \quad \forall i \leq k$. Hago $y_i = f(x_i)$. Combino los y_i en un y que es la solución para x .

El costo de un algoritmo de D&C de tamaño n , se puede expresar como $T(n)$, que debe considerar el costo de hacer la subdivisión de problemas y luego unir los resultados.

No cualquier algoritmo recursivo que divide al problema es divide & conquer.

Las recurrencias de D & C tienen esta forma:

$$T(n) = \begin{cases} \Theta(1) & n \leq k \\ aT(n/c) + f(n) & n > k \end{cases}$$

- **k** es el tamaño del caso base, que en geral vale 1.
- **a** es la cantidad de subproblemas a resolver. "Cant. llamados recursivos".
- **c** es la cantidad de particiones. "Tamaño de los llamados".
- **n/c** es el tamaño de los subproblemas a resolver.
- **f(n)** es el costo de todo lo que se hace en cada llamado además de los llamados recursivos, así, incluye el costo de la división ($D(n)$) y la combinación de los resultados ($C(n)$).

Asumimos que resolver un caso base cuesta $\Theta(1)$.

Teorema maestro:

Si nuestra recurrencia es de la forma:

$$T(n) = \begin{cases} \Theta(1) & n=1 \\ aT(n/c) + f(n) & n>1 \end{cases}$$

con $a \geq 1$ y $c > 1$. Entonces:

$$T(n) = \begin{cases} \Theta(n^{\log_c a}) & ; \text{Si } \exists \varepsilon > 0 / f(n) \in O(n^{\log_c a - \varepsilon}) \\ \Theta(n^{\log_c a} \log n) & ; \text{Si } f(n) \in \Theta(n^{\log_c a}) \\ \Theta(f(n)) & ; \text{Si } \exists \varepsilon > 0 / f(n) \in \Omega(n^{\log_c a + \varepsilon}) \end{cases}$$

y $\exists \delta < 1, \exists n_0 > 0 / \forall n \geq n_0$ se cumple:
 $a \cdot f(n/c) \leq \delta f(n)$

Ejercicio 1). máximo de una montaña de longitud n . Queremos encontrar el máximo, es una secuencia estrictamente creciente seguida de una estrictamente decreciente.

Hay al menos un elemento antes y después del máximo, ya que las secuencias creciente y decreciente tienen al menos 2 elementos.

ej. $[-1, 3, 4, 20, 27, 22, 8, 2, 1]$

casos posibles:

• $A = [-1, 3, 8, 22, 30, 22, 8, 4, 2, 1]$
0 1 2 3 4 5 6 7 8 9



Podemos dividir a la mitad. $c = 2$. Se parte en dos el problema.

$$|A| = 10$$

$$n = 10/2 = 5. \text{ Tomamos elementos del 0 al 5 exclusive.}$$

Me paro en la pos. 5 (el n° 22 en mi montaña) y quiero ver si estoy en la parte creciente o en la decreciente.

caso 1 = $n-1$ es mayor a $n \Rightarrow$ estoy en la parte decreciente.

caso 2 = $n-1$ es menor a $n \Rightarrow$ estoy en la parte creciente.

• $A = [10, 11, 12, -2, -100]$ $n = \frac{5}{2} = 2$ (me quedo con la parte entera de la división).

caso 2. $n-1 < n$
 $11 < 12$

Ejercicio 2). Subsecuencia de suma máxima.

Subsecuencias de n enteros. Quiero encontrar el máximo valor que se puede obtener sumando elementos continuos.

ej. $[3, -1, 4, 8, -2, 2, -7, 5]$

el valor es 14, que se obtiene de $[3, -1, 4, 8]$

Si una secuencia tiene todos números negativos, se entiende que su subsecuencia de suma máxima es la vacía, por lo tanto el valor es 0.

casos posibles.

• $A = [3, -1, 4, 8, -2, 2, -7, 5]$

divido el arreglo a la mitad. $m = 8/2 = 4$ (punto de corte)

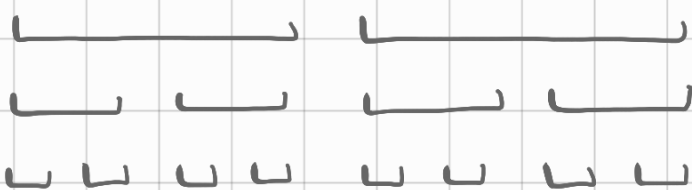
ya no me alcanza con mirar los casos por separado. Tengo que mirar también el medio (porque se me pueden escapar subsecuencias que me importen).

$a = 2 \rightarrow$ voy a tener que mirar las dos partes.

$c = 2 \rightarrow$ dividí en 2.

dentro de cada mitad, vuelvo a partir hasta llegar a un caso base

↳ arreglo de un elemento. En ese caso ese elemento es el valor máximo (de la suma con él mismo).

$$[3, -1, 4, 8, -2, 2, -7, 5]$$


Partición 1.

Partición 2.

Partición 3.

voy comparando: $e^3 \vee -1$? 3

4	✓	8	?	8
---	---	---	---	---

3 v 8 ? 8 Nos quedamos con este.

$[3, -1, 4, 8, -2, 2, -7, 5]$

parriendo del medio. Para la derecha, lo más grande que tenemos es cero $(-2+2)$. Para la izquierda, lo más grande es 14 $(8+4-1+3)$.

(puedo no incluir -2 y 2 porque no modifica nada).

$14 + 0 = 14 \rightarrow$ subsequência máxima.

pseudocódigo:

SumaHaciaDerecha (A) $\rightarrow$ int :

$$\max \text{Suma} = 0$$

$O(1)$

$$\text{sumaAcumulada} = 0$$

0 (1)

```
for i = 0 TO n-1 do
```

 $\# O(n)$

SumaAcumulada += A[i]

$$\text{maxSuma} = \max(\text{maxSuma}, \text{sumaAcumulada})$$

```
return maxSuma
```

↑ Función que analiza las sumas con un pie en el medio.

SumaAlMedio (A) $\rightarrow$ int:

$S_1 = \text{SumaHaciaDerecha}(A[\frac{n}{2} \dots n])$ # $O(n)$

$S_2 = \text{SumaHaciaIzquierda}(A[0 \dots \frac{n}{2}])$ # $O(n)$

return $S_1 + S_2$

SumaSubsecuencia (A) $\rightarrow$ int:

if $|A| = 1$ then

rango de 1 elemento

return $\max(0, A[0])$

Pues si es negativa me
tiene que devolver 0.

$S_1 = \text{SumaSubsecuencia}(A[0 \dots \frac{n}{2}])$

$T(\frac{n}{2})$ } (*)

$S_2 = \text{SumaSubsecuencia}(A[\frac{n}{2} \dots n])$

$T(\frac{n}{2})$

$S_3 = \text{SumaSubsecuencia}$

$O(n)$ pues

debo recorrer todo
el arreglo.

return $\max(S_1, S_2, S_3)$

(*) llamada recursiva donde le pasamos la mitad del arreglo.

Costo: $f(n) = O(n)$

$a = 2$

$c = 2$

$\log_c a = \log_2 2 = 1 \Rightarrow O(n \log n)$ Segundo caso del
teorema maestro.

Heurísticas

Es un procedimiento computacional que intenta obtener soluciones de buena calidad para un problema, intentando que su comportamiento sea lo más preciso posible.

Una heurística para un problema de optimización obtiene una solución con un valor que se espera sea cercano (o igual) al valor óptimo.

Decimos que A es un algoritmo ϵ -aproximado ($\epsilon \geq 0$) para un problema, si
$$\left| \frac{X_A - X_{OPT}}{X_{OPT}} \right| \leq \epsilon$$

Algoritmos Golosos.

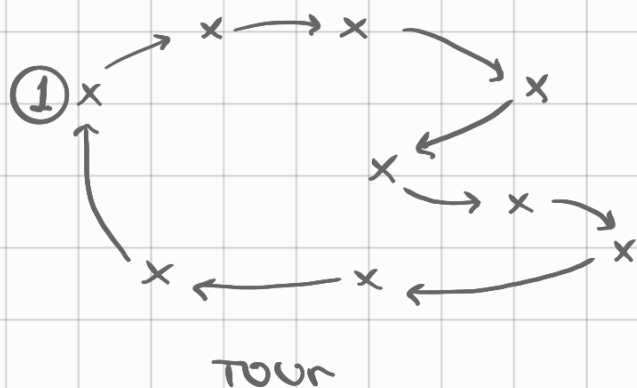
idea: Construir una solución seleccionando en cada caso la mejor alternativa, sin considerar las implicancias de esta selección.

↳ Tomar la mejor solución localmente.

Proporcionar heurísticas sencillas para problemas de optimización.

Ejercicio 1). Problema del viajante de comercio.

Tenemos un conjunto de ciudades. Empezando de una ciudad inicial, queremos recorrerlas a todas y volver a la ciudad inicial con la menor distancia posible.



n ciudades $\Rightarrow n + 1$ tours
distintos posibles.

algoritmo goloso:

arrancamos en una ciudad cualquiera, y me muevo a la más cercana.

Si tiene cruces, probablemente la solución no es óptima.

Este algoritmo es $O(n^2)$, dado que en cada caso debemos buscar entre las ciudades aún no visitadas, la que se encuentre más cerca.

Fuerza bruta vs algoritmos golosos.

	fuerza bruta	algoritmos golosos
calidad de la solución	óptima	sub-óptima
complejidad	alta	eficiente